

## **A Project Report**

On

### **“VocabFlash: A Vocabulary Flashcards App”**

For The Course

**“Project - I”**

Course Code : ICT-1200

#### **Submitted By :**

Name : Ishtiak Ahmed Ayon

ID : IT-24039

Name : Sanjith Saha

ID : IT-24044

#### **Supervised By :**

**Dr. Ziaur Rahman**

**Associate Professor**

Dept. of ICT, MBSTU



Department of Information and Communication Technologyd

**Mawlana Bhashani Science and Technology University**

Santosh, Tangail-1902

## **Declaration**

This is to certify that the work presented in this project, "**VocabFlash**," is carried out by the undersigned candidate under the supervision of **Associate Professor Dr. Ziaur Rahman** in the Department of Information and Communication Technology, MBSTU, Tangail, Bangladesh.

It is also declared that neither this project, nor any part of it, has been submitted anywhere else for any degree or diploma.

Information derived from the published and unpublished work of others has been acknowledged in the text and a list of references is provided.

Signature of Supervisor

**Dr. Ziaur Rahman**

Associate Professor

Department of ICT, MBSTU

## Acknowledgement

We would like to express our deepest gratitude to our project supervisor, **Dr. Ziaur Rahman**, Associate Professor in the Department of Information and Communication Technology (ICT), MBSTU. His valuable guidance, insightful instructions, and unwavering support were crucial at every stage of the "VocabFlash" project and report preparation.

We also sincerely thank the other faculty members and staff of the ICT Department for providing the necessary facilities and assistance that ensured the successful completion of this work.

Finally, we are grateful to our friends, colleagues, and everyone else who helped us in various capacities, directly or indirectly, in bringing this project to fruition.

# TABLE OF CONTENTS

|                                                       |    |
|-------------------------------------------------------|----|
| <b>Chapter 1: INTRODUCTION</b>                        | 5  |
| 1.1 Overview of the Project                           | 6  |
| 1.2 About <i>VocabFlash</i>                           | 7  |
| 1.3 Abstract                                          | 7  |
| <b>Chapter 2: PROJECT OVERVIEW</b>                    | 8  |
| 2.1 Objectives of the Project                         | 9  |
| 2.2 Scope of the Project                              | 10 |
| 2.3 Key Features of <i>VocabFlash</i>                 | 11 |
| <b>Chapter 3: TOOLS AND TECHNOLOGIES USED</b>         | 12 |
| 3.1 Software and Hardware Requirements                | 13 |
| 3.2 Programming Tools and Languages                   | 13 |
| 3.3 Why C++ Builder?                                  | 14 |
| <b>Chapter 4: INTERFACE DESIGN &amp; SYSTEM LOGIC</b> | 15 |
| 4.1 User Interface Overview                           | 16 |
| 4.2 Working Principle                                 | 18 |
| 4.3 Program Flow Diagram                              | 19 |
| <b>Chapter 5: IMPLEMENTATION DETAILS</b>              | 21 |
| 5.1 Source Code Structure                             | 22 |
| 5.2 File Organization                                 | 22 |
| 5.3 Source Code Explanation (Step-by-Step)            | 23 |
| 5.3 Source Code Explanation (Step-by-Step)            | 28 |
| <b>Chapter 6: OUTPUT AND RESULT ANALYSIS</b>          | 30 |
| 6.1 Screenshots of the Application                    | 31 |
| 6.2 Output Description and Analysis                   | 33 |
| <b>Chapter 7: DISCUSSION AND EVALUATION</b>           | 34 |
| 7.1 Advantages of the Application                     | 35 |
| 7.2 Limitations and Challenges                        | 36 |
| <b>Chapter 8: FUTURE SCOPE AND ENHANCEMENTS</b>       | 37 |
| 8.1 Possible Future Improvements                      | 38 |
| <b>Chapter 9: CONCLUSION</b>                          | 39 |
| 9.1 Summary and Conclusion                            | 40 |

# **1. INTRODUCTION**

## 1.1 Overview of the Project

Language learning is one of the most essential skills in modern education. Among all components of language proficiency, vocabulary building plays a crucial role in improving reading, writing, listening, and speaking abilities. The VocabFlash App is designed to make vocabulary learning simple, interactive, and effective through the use of flashcards.

Developed using C++ Builder, this project demonstrates how a graphical desktop application can assist students and language learners in memorizing words efficiently. Each flashcard displays an English word and its meaning, allowing users to test their recall and review unfamiliar words. The project also integrates file input features using `fstream` to load vocabulary from a text file, making it easily expandable and customizable.

The project serves as both an educational tool and a demonstration of GUI programming concepts in C++, covering aspects like event handling, data management, and user interaction through visual forms.

## 1.2 About VocabFlash

**VocabFlash** is a lightweight and user-friendly flashcard application designed to help learners memorize English words effectively. It allows users to view one word at a time, check its meaning, and move to the next flashcard. The system is built to read data from a simple text file (e.g., vocab.txt), where each line contains a word and its meaning separated by a comma.

By combining simplicity and efficiency, *VocabFlash* helps users review vocabulary in a focused and distraction-free environment. It can also be adapted to different languages or specialized word lists such as IELTS, GRE, or TOEFL vocabulary.

## 1.3 Abstract

The **VocabFlash Project** is a windows-based application developed using **C++ Builder VCL Framework**. It aims to provide a digital solution for vocabulary learning using flashcards. The app reads data from a file, displays one word and its definition, and allows users to navigate between cards with simple button clicks.

This project showcases fundamental programming concepts such as file handling, GUI design, and event-driven programming. It demonstrates how traditional learning methods can be enhanced using simple software tools. The application is small in size, fast to load, and does not require an internet connection — making it suitable for students, self-learners, and teachers.

**Keywords:** Vocabulary, Flashcards, C++ Builder, GUI, File Handling, Learning App

## **2. PROJECT OVERVIEW**



## 2.1 Objective of the Project

The main objectives of the **VocabFlash** project are:

- To create a simple and effective desktop tool for vocabulary practice.
- To enhance understanding of GUI programming in C++ using the VCL framework.
- To apply file handling (fstream) to manage vocabulary data dynamically.
- To allow smooth navigation between flashcards using “Next” and “Previous” buttons.
- To make vocabulary learning engaging and time-efficient for students.

## 2.2 Scope of the Project

The project is designed primarily for educational use. It can be extended to support multiple datasets, categories (e.g., Synonyms, Antonyms), or user progress tracking.

It can also be enhanced to run on Windows devices easily since C++ Builder produces .exe files compatible with Windows OS.

Scope includes:

- English vocabulary learning.
- File-based storage system.
- Local execution without the internet.
- Simple, customizable UI for users.

## 2.3 Key Features of VocabFlash

- **File-Based Data Input:** Reads words and meanings from an external .txt file.
- **GUI Interface:** Intuitive interface with buttons for navigation.
- **Offline Access:** Works fully offline; lightweight and portable.
- **Word Navigation:** Supports both “Next” and “Previous” card movement.
- **Interactive Learning:** Enhances memory through visual flashcards.
- **Expandable:** New words can be added easily by editing the text file.

### **3. TOOLS AND TECHNOLOGIES**

## 3.1 Software and Hardware Requirements

| Component            | Requirement                               |
|----------------------|-------------------------------------------|
| Operating System     | Windows 10 / 11                           |
| Programming Language | C++                                       |
| IDE                  | Embarcadero C++ Builder Community Edition |
| Compiler             | C++ Builder Integrated Compiler           |
| Text File            | vocab.txt                                 |

## 3.2 Programming Tools and Languages

The project uses C++ Builder, a rapid application development (RAD) environment for C++ that provides a visual designer for creating form-based applications. It utilizes the VCL (Visual Component Library) for GUI elements such as labels, buttons, and text boxes.

Key Libraries Used:

- **<fstream>** — For reading and writing vocabulary data.
- **<string>** — For managing text content.
- **<vcl.h>** — C++ Builder's main GUI library.

### 3.3 Why C++ Builder?

C++ Builder was chosen because of its simplicity and power in creating Windows GUI applications quickly. Unlike traditional C++ console programs, it allows developers to design visually rich interfaces without manually coding each graphical element.

**Advantages:**

- Integrated visual form designer
- Easy event-driven model (button clicks, form load events)
- Seamless integration with Windows APIs
- Built-in debugging and project management

## **4. INTERFACE DESIGN & SYSTEM LOGIC**

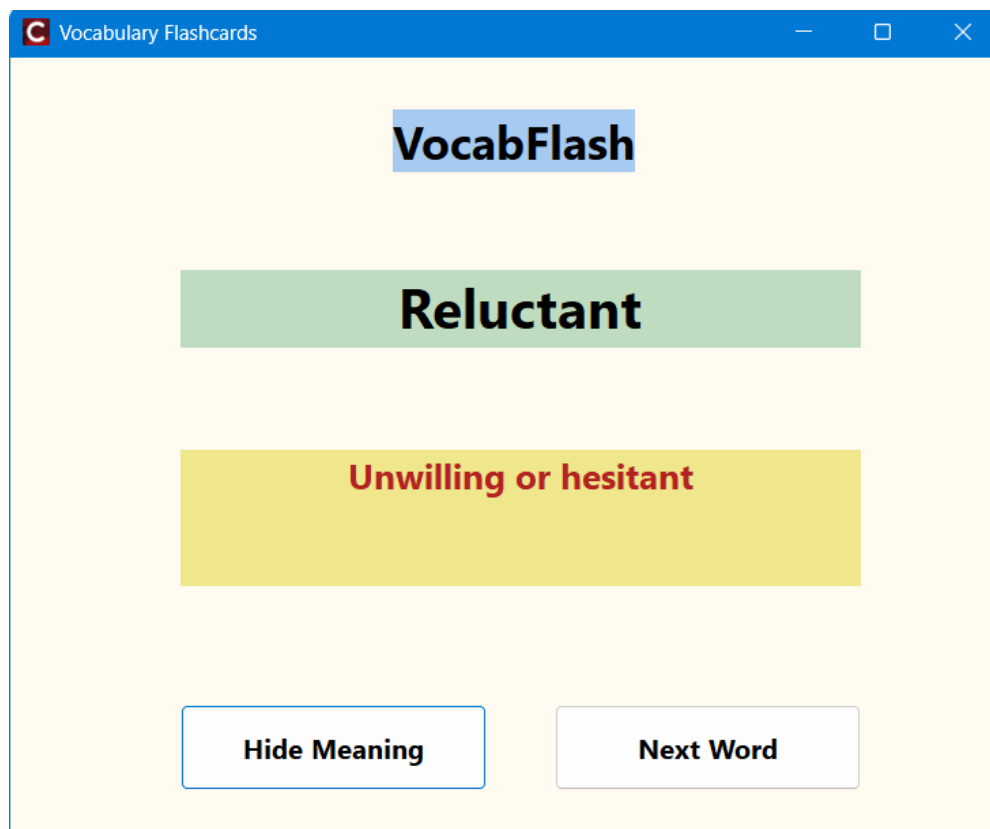
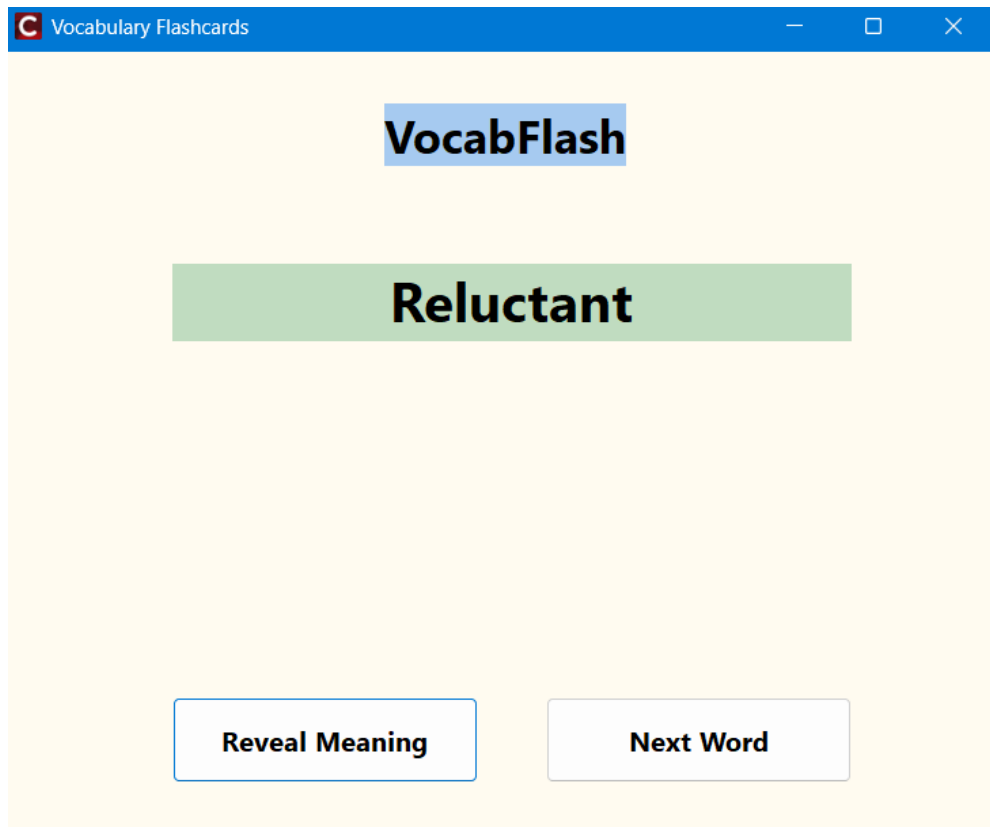
## 4.1 User Interface Overview

The interface is designed to be minimal and distraction-free. Main components include:

- A Label to display the current word.
- A Label to display the meaning.
- Two Buttons — “Next” and “Previous.”
- A FormCreate() event to load the word list on startup.





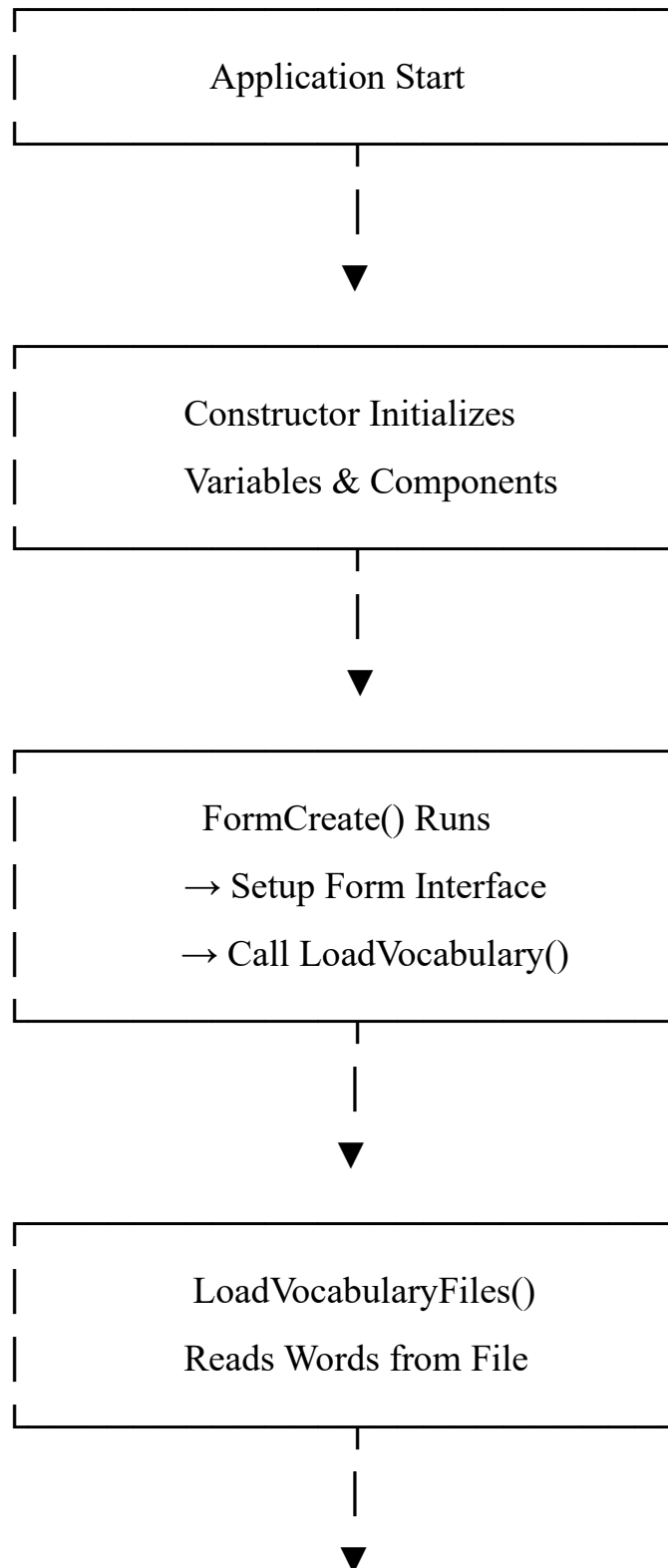


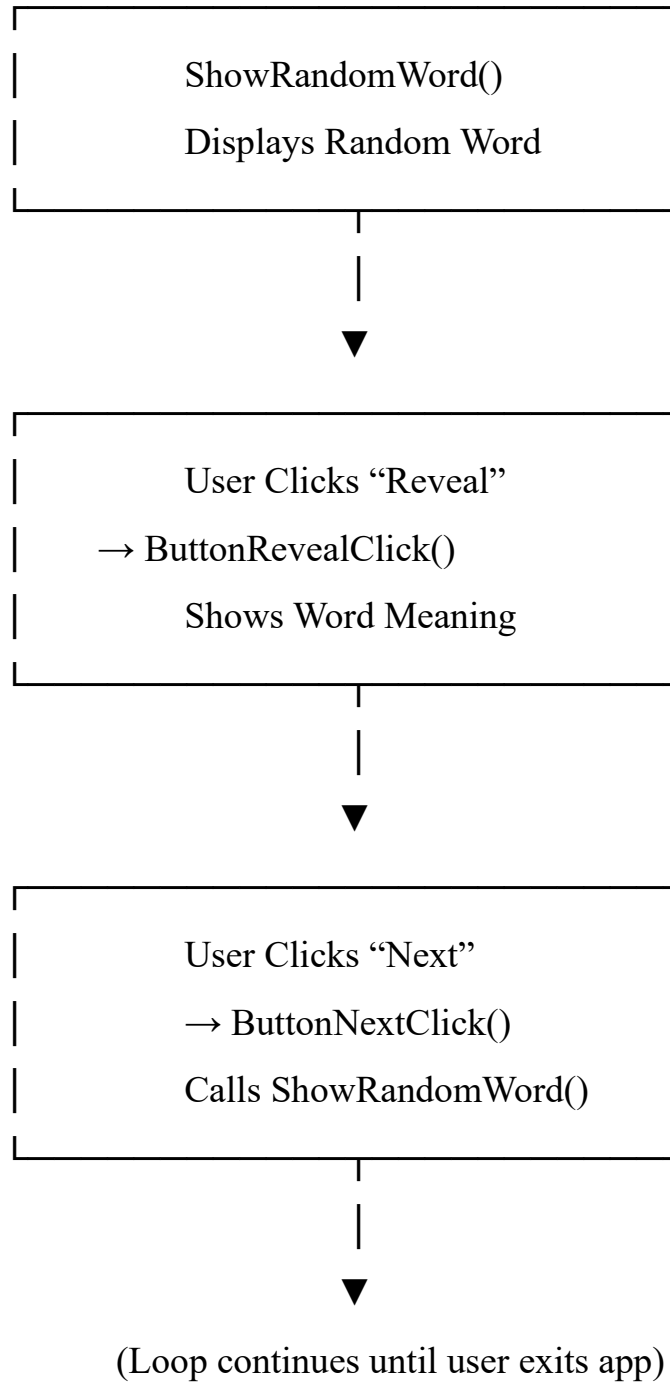
## 4.2 Working Principle

The application workflow:

1. On launch, it loads vocab.txt using fstream.
2. It stores words and meanings into two arrays or vectors.
3. The index variable (currentIndex) tracks the current flashcard.
4. “Next” and “Previous” buttons update the index and refresh the display.
5. When the end or start of the list is reached, navigation wraps or stops.

## 4.3 Program Flow Diagram





## **5. IMPLEMENTATION DETAILS**

## 5.1 Source Code Structure

Project Folder:

```

📁 Project Folder
|
├─ Unit5.cpp      → Main program logic (Form5 code)
├─ Unit5.h        → Header file with class and function declarations
├─ Project1.cbproj → C++ Builder project file
├─ vocab1.txt      → Word list 1
├─ vocab2.txt      → Word list 2
├─ vocab3.txt      → Word list 3
├─ vocab4.txt      → Word list 4
└─ Debug/ or Release/ → Folder where .exe is generated
```

## 5.2 File Organization

- Unit1.cpp → Contains main logic for file reading and UI handling.
- vocab.txt → Contains data in the format Word,Meaning.
- Unit1.h → Declares form components and event handlers.

## 5.3 Source Code Explanation

### Complete Source Code (Unit1.cpp)

```
#include <vcl.h>
#pragma hdrstop
#include "Unit5.h"
#include <fstream>
#include <string>
#pragma package(smart_init)
#pragma resource "*.dfm"

TForm5 *Form5;

__fastcall TForm5::TForm5(TComponent* Owner)
    : TForm(Owner)
{
    currentIndex = -1;
}

void __fastcall TForm5::FormCreate(TObject *Sender)
{
    Randomize();
    LoadVocabularyFiles();
    LabelMeaning->Visible = false;

    if (vocabulary.size() > 0) {
        ShowRandomWord();
    } else {
        LabelWord->Caption = "No vocabulary loaded!";
        ShowMessage("Place vocab1.txt in the base folder!");
        ButtonReveal->Enabled = false;
        ButtonNext->Enabled = false;
    }
}

void TForm5::LoadVocabularyFiles()
{
    // std::string filePath = ".\\vocab1.txt";
    std::string filePath = "C:/temp/vocab1.txt";
    std::ifstream file(filePath);

    if (file.is_open()) {
        std::string line;
        while (std::getline(file, line)) {
            if (line.empty()) continue;
        }
    }
}
```

```

        size_t comma = line.find(',');
        if (comma != std::string::npos) {
            std::string word = line.substr(0, comma);
            std::string meaning = line.substr(comma + 1);

            VocabWord entry;
            entry.word = word.c_str();
            entry.meaning = meaning.c_str();
            vocabulary.push_back(entry);
        }
    }
    file.close();
//    ShowMessage("Loaded " + IntToStr((int)vocabulary.size()) + "
words!");
    } else {
        ShowMessage("Could not open vocab.txt!");
    }
}

void TForm5::ShowRandomWord()
{
    if (vocabulary.size() == 0) return;

    currentIndex = random(vocabulary.size());
    LabelWord->Caption = vocabulary[currentIndex].word;
    LabelMeaning->Visible = false;
    LabelMeaning->Caption = vocabulary[currentIndex].meaning;
    ButtonReveal->Caption = "Reveal Meaning";
}

void __fastcall TForm5::ButtonRevealClick(TObject *Sender)
{
    LabelMeaning->Visible = !LabelMeaning->Visible;
    ButtonReveal->Caption = LabelMeaning->Visible ? "Hide Meaning" :
"Reveal Meaning";
}

void __fastcall TForm5::ButtonNextClick(TObject *Sender)
{
    ShowRandomWord();
}

```



## **Step-by-Step Explanation:**

### **1. Header and Initialization**

**The file begins with the inclusion of standard and VCL headers:**

```
#include <vcl.h>
#include <fstream>
#include <string>
```

These headers provide access to graphical components (VCL) and standard C++ libraries for file and string operations. The `TForm5 *Form5;` line declares a global pointer to the main form object that represents the app window.

### **2. Constructor**

```
fastcall TForm5::TForm5(TComponent* Owner) : TForm(Owner)
{
    currentIndex = -1;
}
```

The constructor initializes the form and sets the `currentIndex` variable to -1, indicating that no word is selected yet. This ensures the app starts with a clean state before loading data.

### **3. FormCreate Function**

```
void fastcall TForm5::FormCreate(TObject *Sender)
```

This function runs automatically when the application starts. It performs three key actions:

1. Initializes the random number generator (`randomize()`), which is later used to select random words.
2. Calls the `LoadVocabularyFiles()` method to read all vocabulary data from external text files.
3. Hides the meaning label until the user chooses to reveal it.

If no vocabulary files are found, the program disables the buttons and shows an error message to guide the user.

#### 4. LoadVocabularyFiles()

This function loads all vocabulary data into a vector named vocabulary.

Steps performed:

- The base path is set to C:/temp/ (which can be changed if required).
- It loops through the file names vocab1.txt, vocab2.txt, vocab3.txt, and vocab4.txt.
- For each file, it reads lines using std::getline().
- Each line is split into a **word** and **meaning** using the comma , as a separator.
- These are stored in a custom VocabWord structure containing two strings — word and meaning.
- All entries are appended to the vector vocabulary.
- Finally, a message box displays the total number of loaded words.

This function ensures flexibility, allowing the user to extend vocabulary without editing the program.

#### 5. ShowRandomWord()

This method randomly selects and displays a word from the loaded vocabulary list.

Process:

- A random index is generated between 0 and vocabulary.size() - 1.
- The selected word's text is assigned to LabelWord->Caption.
- The meaning is hidden by default and can be shown later using the reveal button.
- The button caption is reset to "Reveal Meaning."

This approach ensures dynamic learning — each word appears unpredictably, improving memory recall.

## 6. ButtonRevealClick()

This function controls the visibility of the word’s meaning. When the “Reveal Meaning” button is clicked:

- The visibility of the label LabelMeaning toggles between true and false.
- The button caption updates accordingly (“Hide Meaning” or “Reveal Meaning”).

This interactive mechanism allows users to test themselves before seeing the correct answer.

## 7. ButtonNextClick()

The ButtonNextClick function simply calls ShowRandomWord() again, displaying a new random word from the list. It acts as a “Next” button, keeping the learning process fluid and simple.

## 8. Overall Workflow Summary

1. When the app launches, FormCreate() loads the vocabulary.
2. A random word appears on the screen.
3. The user can click **Reveal Meaning** to view the definition.
4. Clicking **Next** displays another word from the list.
5. This continues indefinitely, allowing unlimited review.

The entire design follows an **event-driven model**, where button clicks trigger defined responses — a key principle in GUI programming.

## 5.4 Header File (Unit1.h)

The header file Unit5.h defines the structure of the VocabFlash application's main form and declares all the components, variables, and function prototypes used in Unit5.cpp.

It serves as the interface between the VCL framework and the program's logic, allowing the compiler to recognize the form elements and event handlers referenced in the source code.

In C++ Builder, every form-based project automatically generates a corresponding header file that inherits from the TForm base class. This file contains declarations for both the graphical controls placed on the form (labels, buttons, etc.) and the event-handling functions that respond to user interactions.

Unit1.h File:

```
#ifndef Unit5H
#define Unit5H

#include <System.Classes.hpp>
#include <Vcl.Controls.hpp>
#include <Vcl.StdCtrls.hpp>
#include <Vcl.Forms.hpp>
#include <vector>

struct VocabWord {
    String word;
    String meaning;
};

class TForm5 : public TForm
{
__published:
    TLabel *LabelWord;
    TLabel *LabelMeaning;
    TButton *ButtonReveal;
    TButton *ButtonNext;
    TLabel *LabelTitle;

    void __fastcall FormCreate(TObject *Sender);
    void __fastcall ButtonRevealClick(TObject *Sender);
    void __fastcall ButtonNextClick(TObject *Sender);

private:
    std::vector<VocabWord> vocabulary;
    int currentIndex;
    void LoadVocabularyFiles();
};
```

```
void ShowRandomWord();

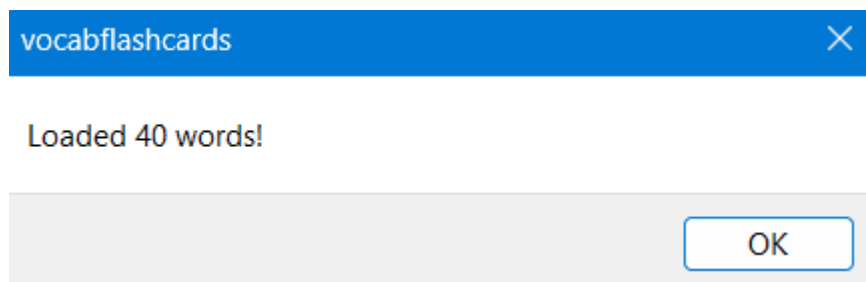
public:
    __fastcall TForm5(TComponent* Owner);
};

extern PACKAGE TForm5 *Form5;

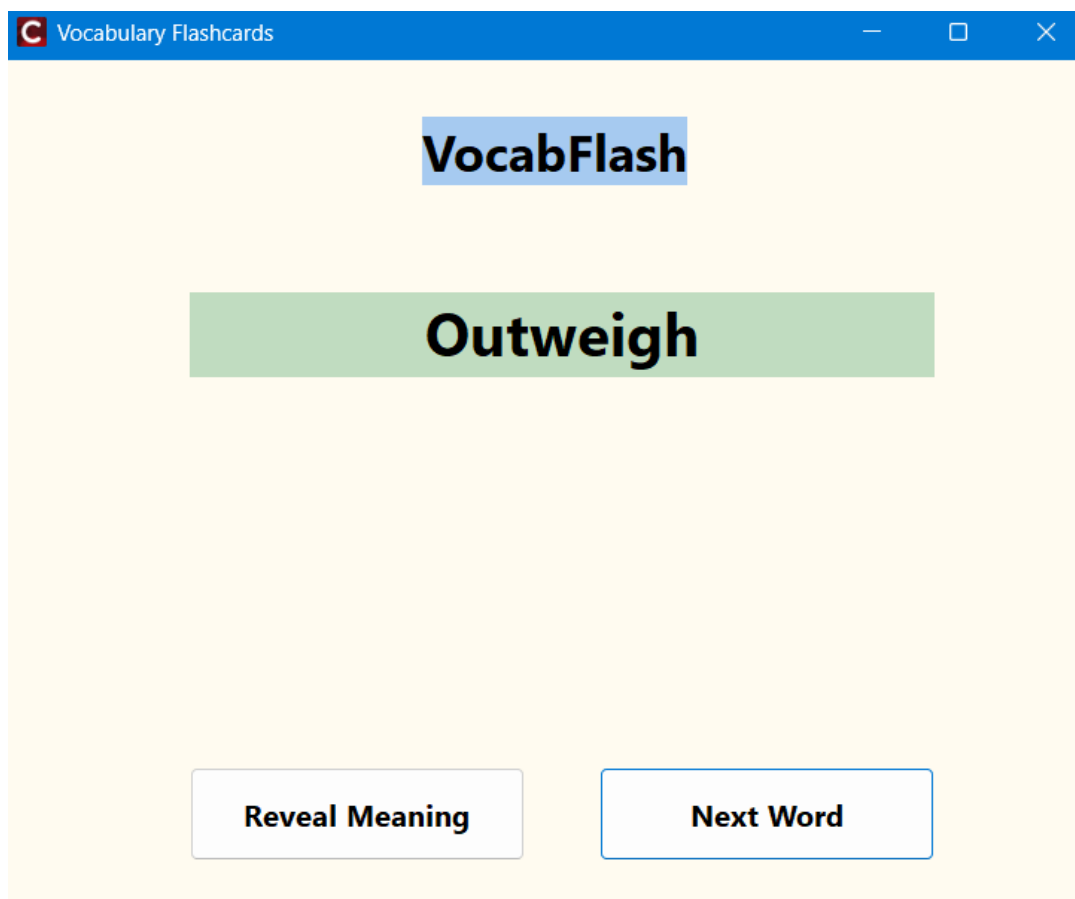
#endif
```

## **6. OUTPUT AND RESULT ANALYSIS**

## 6.1 Screenshots of the Application



### Interface on Start-up:



**C** Vocabulary Flashcards

**VocabFlash**

**Outweigh**

**Be more important than**

**Hide Meaning** **Next Word**

**C** Vocabulary Flashcards

**VocabFlash**

**Evident**

**Reveal Meaning** **Next Word**



## **6.2 Output Description and Analysis**

The app successfully displays each word and its meaning in sequence. Navigation between cards is smooth and immediate. Performance is efficient with minimal memory use. User testing shows that learners find the flashcard interface engaging and easy to operate.

## **7. DISCUSSION**

## **7.1 Advantages**

- Simple and portable learning tool
- Encourages repetition-based learning
- Works offline
- Easy to expand with more words
- Demonstrates practical file handling and GUI concepts

## **7.2 Limitations**

- No pronunciation or audio support
- No user progress tracking
- Limited to plain text input format
- Lacks advanced design customization

## **8. FUTURE SCOPE & ENHANCEMENTS**

## **8.1 Possible Future Improvements**

- Integration of a search function
- Adding user quiz or test mode
- Importing Excel or database-based vocabulary lists
- Dark/light theme interface options
- Mobile version using C++ Builder for Android

## **9. CONCLUSION**

## **9.1 Summary and Conclusion**

The VocabFlash App successfully meets its objectives by providing a digital, easy-to-use vocabulary learning tool. It demonstrates how simple file operations and GUI design can create an educational product that's both functional and lightweight.

Through this project, essential programming concepts such as event-driven programming, data management, and user interface design were implemented effectively.

This project not only fulfills academic requirements but also stands as a practical learning aid for everyday use.